

Deploying a Kedro Project as a REST API on Cloud Run via Docker and GitHub Actions

This guide shows how to convert a **Kedro pipeline** into a **REST API** using **FastAPI** and package it into a **Docker** container. It then walks through configuring **CI/CD with GitHub Actions** to build and deploy the container to **Google Cloud Run**, using **Artifact Registry** as the container registry. The workflow emphasises reproducibility, code quality (ruff + black), and experiment tracking with **MLflow**.

1. Mental model

Kedro is designed for reproducible pipelines, not for serving APIs. The minimal bridging pattern is:

1. **Validate request** with FastAPI/Pydantic. Extract inputs and convert them into a form your Kedro pipeline expects (e.g., `params` or DataCatalog entries).
2. **Create a Kedro session** and run the pipeline programmatically. Use `pipeline_name` / `tags` to select the correct pipeline and pass `extra_params` to override parameters at runtime. The session API supports `pipeline_name`, `tags`, and `extra_params` arguments ¹.
3. **Return JSON** containing run metadata or outputs. Only lightweight outputs should be returned directly; large artifacts should be stored in a data store (e.g., GCS) referenced in the response.
4. **Reproducibility gates**: local development → pre-commit (lint/format/test) → CI checks → Docker build → deploy. Version your code and track experiments with MLflow. Avoid baking data or secrets into the container image; use environment variables and Secret Manager instead. Bind the app to `0.0.0.0` and respect Cloud Run's `PORT` environment variable so that the container becomes ready ² ³.

2. Repository layout

Assume the repository is named `<project>` and the Python package is `<pkg>`. A typical Kedro layout augmented for an API looks like this:

```
<project>/
├── .dockerignore
├── .github/
│   └── workflows/
│       └── deploy.yml           # GitHub Actions workflow
├── data/                       # optional local data (ignored in Docker)
├── logs/                       # pipeline logs
├── notebooks/
├── pyproject.toml
└── README.md
```

```

├── scripts/                # helper scripts (optional)
├── Dockerfile
├── src/
│   └── <pkg>/
│       ├── __init__.py
│       ├── api/
│       │   └── main.py      # FastAPI application
│       ├── hooks.py        # Kedro MLflow hooks
│       ├── pipelines/
│       │   └── ...         # your pipelines and nodes
│       ├── settings.py     # Kedro settings for hook registration
│       └── ...

```

`src/<pkg>/api/` contains your FastAPI code. The `hooks.py` file registers MLflow hooks, and `settings.py` tells Kedro to use them. The `.github/workflows/deploy.yml` file defines CI/CD.

3. Local development with uv

Use **uv** as the package manager and virtual-environment tool. It writes dependencies into `pyproject.toml` and resolves them deterministically. Initial setup:

```

# create a virtual environment
uv venv .venv
# activate it (bash/zsh)
source .venv/bin/activate

# add core dependencies
uv add --group api fastapi "uvicorn[standard]" # REST framework
uv add --group ml mlflow                      # experiment tracking
uv add --group dev ruff                       # linter
uv add --group dev black                      # formatter
uv add --group dev pytest                     # tests
# add Kedro itself (core + extras for GCP datasets as needed)
uv add kedro kedro-datasets[gcs,bigquery]

```

`uv add --group` writes dependencies into appropriate `dependency-groups` in `pyproject.toml`⁴. You can run commands within the environment using `uv run`, which ensures the environment is up-to-date and available⁵. Example:

```

# run Kedro tests
uv run pytest -q

# lint code with ruff
uv run ruff check src/ tests/

```

```
# format check with black
uv run black --check src/ tests/
```

If you need to generate a traditional `requirements.txt` for Docker, `uv pip freeze > requirements.txt`, but the recommended approach is to run `uv` inside the container (see Dockerfile).

A minimal `pyproject.toml` snippet with tool configuration:

```
[project]
name = "<pkg>"
version = "0.1.0"
requires-python = ">=3.10"

[tool.uv.dependency-groups]
dev = ["ruff", "black", "pytest"]
api = ["fastapi", "uvicorn[standard]"]
ml = ["mlflow"]

[tool.ruff]
line-length = 100
select = ["E", "F", "B"] # basic error/flake and bugbear rules

[tool.black]
line-length = 100
skip-string-normalization = true
```

Ruff automatically discovers its configuration in `pyproject.toml` ⁶, and Black's `--check` option can be used to ensure files are formatted without modifying them ⁷.

4. Turning a Kedro pipeline into a service

Minimal pipeline example

Create a simple Kedro pipeline in `src/<pkg>/pipelines/example/` with three nodes: load data, train model, and predict. Register it in `pipelines.py` as the default pipeline.

```
# src/<pkg>/pipelines/example/nodes.py
from sklearn.linear_model import LinearRegression

def load_data() -> tuple[list[float], list[float]]:
    # in practice, read from the catalog
    X = [1, 2, 3, 4]
    y = [2, 4, 6, 8]
```

```

    return X, y

def train_model(data: tuple[list[float], list[float]]) -> LinearRegression:
    X, y = data
    model = LinearRegression().fit([[x] for x in X], y)
    return model

def predict(model: LinearRegression, x_new: list[float]) -> list[float]:
    return [float(model.predict([[x]])[0]) for x in x_new]

```

In `pipelines/example/pipeline.py` define the pipeline:

```

from kedro.pipeline import node, Pipeline
from . import nodes

def create_pipeline() -> Pipeline:
    return Pipeline([
        node(nodes.load_data, outputs="example_data", name="load_data"),
        node(nodes.train_model, inputs="example_data", outputs="example_model",
name="train_model"),
        node(nodes.predict, inputs=dict(model="example_model",
x_new="params:x_new"), outputs="predictions", name="predict"),
    ])

```

Running the pipeline programmatically

In a normal CLI run, Kedro reads parameters from YAML files. For an API, call the pipeline from Python. Use `KedroSession.create()` and call `session.run()` specifying the pipeline name or tags. Pass dynamic parameters via `extra_params` – these override configuration defaults ¹:

```

from kedro.framework.session import KedroSession

with KedroSession.create(package_name="<pkg>", env="prod") as session:
    outputs = session.run(
        pipeline_name="example", # or tags=[...]
        extra_params={"x_new": [10, 20, 30]},
    )
    predictions = outputs["predictions"]

```

`session.run()` returns a dictionary containing outputs from nodes not written to the DataCatalog ¹. Avoid global state; create a new session per request. If you need to isolate resource usage or concurrency, run pipelines asynchronously in a thread pool.

5. FastAPI application

Create `src/<pkg>/api/main.py` for the REST API. The app exposes a health check and a run endpoint. It validates input with Pydantic, calls the Kedro pipeline, logs the run, and returns predictions.

```
# src/<pkg>/api/main.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import logging
import os
from kedro.framework.session import KedroSession

# Configure logging
logging.basicConfig(level=os.environ.get("LOG_LEVEL", "INFO"))
logger = logging.getLogger(__name__)

app = FastAPI(title="Kedro Service API")

class RunRequest(BaseModel):
    x_new: list[float]

class RunResponse(BaseModel):
    run_id: str
    predictions: list[float]

@app.get("/health")
def health() -> dict[str, str]:
    return {"status": "ok"}

@app.post("/run", response_model=RunResponse)
def run(request: RunRequest) -> RunResponse:
    try:
        with KedroSession.create(package_name="<pkg>",
env=os.getenv("KEDRO_ENV", "prod")) as session:
            outputs = session.run(
                pipeline_name="example",
                extra_params={"x_new": request.x_new},
            )
            preds = outputs["predictions"]
            run_id = session.session_id
    except Exception as exc:
        logger.exception("Pipeline execution failed")
        raise HTTPException(status_code=500, detail=str(exc))
    return RunResponse(run_id=run_id, predictions=preds)
```

Key points:

- The server binds to `0.0.0.0` and reads the port from the `PORT` environment variable when launched (see Dockerfile). Cloud Run injects `PORT` and expects the app to listen on all interfaces ².
- Wrap `KedroSession.create()` in a context manager to ensure proper teardown ⁸.
- Use Pydantic models to validate input and output.
- Add structured logging; include `run_id` in log messages for traceability.

6. MLflow integration

A robust way to track experiments is via a **Kedro hook**. Define hooks in `src/<pkg>/hooks.py` to start and end an MLflow run around pipeline execution and log parameters/metrics:

```
# src/<pkg>/hooks.py
from typing import Any, Dict
import mlflow
from kedro.framework.hooks import hook_impl
from kedro.pipeline.node import Node

class MLflowHooks:
    """Log run metadata, parameters and model artifacts to MLflow."""

    @hook_impl
    def before_pipeline_run(self, run_params: Dict[str, Any]) -> None:
        # Start an MLflow run with the Kedro run_id
        mlflow.start_run(run_name=run_params["run_id"])
        mlflow.log_params(run_params)

    @hook_impl
    def after_node_run(self, node: Node, outputs: Dict[str, Any], inputs:
Dict[str, Any]) -> None:
        # Example: log split ratio and model
        if node.name == "train_model":
            model = outputs["example_model"]
            mlflow.sklearn.log_model(model, artifact_path="model")
        if node.name == "load_data":
            mlflow.log_params({"num_records": len(inputs["example_data"][0])})

    @hook_impl
    def after_pipeline_run(self) -> None:
        mlflow.end_run()
```

The Kedro documentation provides a similar example showing how hooks start an MLflow run with the `run_id`, log parameters in `before_pipeline_run`, log model and metrics in `after_node_run`, and end the run at the end ⁹. Use environment variables `MLFLOW_TRACKING_URI` and

`MLFLOW_EXPERIMENT_NAME` in your CI/CD and Cloud Run deployment to direct logs to a remote MLflow server. When using MLflow in Cloud Run, the service must have credentials to write to the remote tracking server (e.g., via a service account or token).

Register the hooks in Kedro by defining a `hooks` entry in `src/<pkg>/settings.py`:

```
from <pkg>.hooks import MLflowHooks

HOOKS = (
    MLflowHooks(),
)
```

7. Linting and formatting

Use **ruff** for linting and **black** for formatting. Add the following configuration to `pyproject.toml`:

```
[tool.ruff]
line-length = 100
select = ["E", "F", "B"] # error, flake8 and bugbear
ignore = ["E501"]        # ignore line-length errors if desired

[tool.black]
line-length = 100
skip-string-normalization = true
```

Run checks locally using uv:

```
uv run ruff check src/ tests/
uv run black --check src/ tests/
```

Ruff will exit with non-zero status if any lint errors are found ¹⁰. Black's `--check` flag ensures files are correctly formatted without modifying them ⁷.

8. Dockerfile (production-ready)

Use a multi-stage build to keep the final image small. Use a minimal base image (e.g., `python:3.10-slim` or a distroless Python base). The container must run as a non-root user and listen on `0.0.0.0`. Here is an example `Dockerfile` using uv for dependency resolution:

```
# syntax=docker/dockerfile:1

# Stage 1 - build dependencies
```

```

FROM python:3.10-slim AS builder

# Install uv and build dependencies
RUN apt-get update && apt-get install -y git gcc && rm -rf /var/lib/apt/lists/*
RUN pip install --no-cache-dir uv

WORKDIR /app
COPY pyproject.toml .
# Install production dependencies (api + ml + Kedro + extras) into a virtual env
in /opt/venv
RUN python -m uv pip install --upgrade pip && \
    uv pip install . --group api --group ml --group default --only-deps

# Stage 2 - runtime
FROM python:3.10-slim

ENV PYTHONUNBUFFERED=1
ENV PORT=8080

# Create a non-root user
RUN useradd -m appuser
WORKDIR /app

# Copy virtual environment from builder
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Copy source code
COPY src/ ./src/
COPY src/<pkg>/api ./src/<pkg>/api
COPY src/<pkg>/pipelines ./src/<pkg>/pipelines
COPY src/<pkg>/hooks.py ./src/<pkg>/hooks.py
COPY src/<pkg>/settings.py ./src/<pkg>/settings.py

# Copy Kedro configuration (conf, catalog, parameters)
COPY conf/ ./conf/

# Use an unprivileged user
USER appuser

# Start the FastAPI app. Use PORT env variable provided by Cloud Run
CMD ["uvicorn", "<pkg>.api.main:app", "--host", "0.0.0.0", "--port", "${PORT}"]

```

Key practices:

- Use `.dockerignore` to exclude `data/`, `logs/`, `.venv/`, `notebooks/`, etc. Only include code and configuration necessary at runtime.

- Avoid copying secrets or large datasets into the image.
- Respect `PORT` and `0.0.0.0` to ensure Cloud Run readiness ² ³ .

9. GCP setup

Perform these steps once per project; replace placeholders (`<PROJECT_ID>`, `<REGION>`, etc.) with your values.

Enable APIs

```
gcloud services enable artifactregistry.googleapis.com run.googleapis.com
secretmanager.googleapis.com
```

Create Artifact Registry repository

DataCamp's Cloud Run tutorial shows how to create a Docker repository in Artifact Registry ¹¹ . Use:

```
# Create a Docker repo named <REPO> in a region
REGION=<REGION>
REPO=<REPO>
PROJECT_ID=<PROJECT_ID>

gcloud artifacts repositories create "$REPO" \
  --repository-format=docker \
  --location="$REGION"

# Configure docker auth for the region
gcloud auth configure-docker "$REGION"-docker.pkg.dev
```

Build and push image manually (optional)

If testing locally, tag and push your image. Tagging and pushing commands are summarised in DataCamp's guide ¹² :

```
docker tag <local-image>:latest "${REGION}-docker.pkg.dev/${PROJECT_ID}/${REPO}/<pkg>:v1"
docker push "${REGION}-docker.pkg.dev/${PROJECT_ID}/${REPO}/<pkg>:v1"
```

Deploy to Cloud Run via gcloud

Deploy the image with `gcloud run deploy` using the repository URI. DataCamp's example shows the command and explains flags ¹³ :

```
gcloud run deploy <SERVICE> \
  --image "${REGION}-docker.pkg.dev/${PROJECT_ID}/${REPO}/<pkg>:v1" \
  --platform managed \
  --region "$REGION" \
  --allow-unauthenticated
```

`--allow-unauthenticated` exposes the service publicly; remove it to require IAM authentication. After deployment, Cloud Run returns the service URL.

IAM and Workload Identity Federation

To allow GitHub Actions to deploy, you need a service account with appropriate roles. The `google-github-actions/deploy-cloudrun` action documentation states that the deploying account needs **Cloud Run Admin** (`roles/run.admin`) and to be a **Service Account User** on the runtime account ¹⁴. A recommended approach is to use **Workload Identity Federation** rather than storing static JSON keys. The `google-github-actions/auth` documentation outlines the steps:

1. Create a Workload Identity Pool:

```
gcloud iam workload-identity-pools create "github" \
  --project="${PROJECT_ID}" \
  --location="global" \
  --display-name="GitHub Actions Pool" 15
```

2. Get the pool ID ¹⁶:

```
gcloud iam workload-identity-pools describe "github" \
  --project="${PROJECT_ID}" \
  --location="global" \
  --format="value(name)"
# returns projects/PROJECT_NUMBER/locations/global/workloadIdentityPools/
github
```

3. Create an OIDC provider restricted to your GitHub organisation or repository ¹⁷:

```
GITHUB_ORG=<your-github-org>
gcloud iam workload-identity-pools providers create-oidc "github-provider" \
  --project="${PROJECT_ID}" \
  --location="global" \
  --workload-identity-pool="github" \
  --display-name="GitHub provider" \
```

```
--attribute-
mapping="google.subject=assertion.sub,attribute.repository=assertion.repository,attribute.re
\
--attribute-condition="assertion.repository_owner=='${GITHUB_ORG}'" \
--issuer-uri="https://token.actions.githubusercontent.com"
```

4. **Create a service account** for deployments and grant it roles. It needs `roles/run.admin` and `roles/artifactregistry.writer`. Then bind the Workload Identity Pool to act as that service account ¹⁸ :

```
SA_NAME=github-actions
gcloud iam service-accounts create "$SA_NAME" --project "$PROJECT_ID"

# grant Cloud Run Admin and Artifact Registry Writer roles to the service
account
gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${SA_NAME}@${PROJECT_ID}.iam.gserviceaccount.com" \
  --role="roles/run.admin"
gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${SA_NAME}@${PROJECT_ID}.iam.gserviceaccount.com" \
  --role="roles/artifactregistry.writer"

# allow the workload identity pool to impersonate the service account
gcloud iam service-accounts add-iam-policy-binding "${SA_NAME}@${PROJECT_ID}.iam.gserviceaccount.com" \
  --project="${PROJECT_ID}" \
  --role="roles/iam.workloadIdentityUser" \
  --member="principalSet://iam.googleapis.com/${WORKLOAD_IDENTITY_POOL_ID}/attribute.repository/${GITHUB_ORG}/${REPO_NAME}"
```

5. Use the generated `workload_identity_provider` and service account email in GitHub Actions.

Alternatively, you can generate a JSON key and store it in a GitHub secret, but this is less secure and should be avoided ¹⁹ .

10. GitHub Actions CI/CD

Create `.github/workflows/deploy.yml` to build, test, and deploy. This example uses Workload Identity Federation for authentication and the official `deploy-cloudrun` action. Replace placeholders accordingly.

```

name: CI and Deploy
on:
  push:
    branches: [main]

permissions:
  contents: read
  id-token: write      # needed for Workload Identity Federation

env:
  PROJECT_ID: ${ secrets.GCP_PROJECT_ID }
  REGION: ${ secrets.GCP_REGION }
  REPO: ${ secrets.GAR_REPO }
  SERVICE: ${ secrets.CLOUD_RUN_SERVICE }
  PACKAGE: <pkg>

jobs:
  build-test-deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.10'

      - name: Install uv
        run: pip install uv

      - name: Install dependencies
        run: |
          uv venv .venv
          source .venv/bin/activate
          uv pip install . --group api --group ml --group dev --group default

      - name: Lint
        run: |
          source .venv/bin/activate
          uv run ruff check src/
          uv run black --check src/

      - name: Test
        run: |
          source .venv/bin/activate
          uv run pytest -q

```

```

- name: Build Docker image
  run: |
    docker build -t "${{ env.REGION }}-docker.pkg.dev/${{ env.PROJECT_ID }}/${{ env.REPO }}/${{ env.PACKAGE }}:${{ github.sha }}" .

- name: Authenticate to Google Cloud
  uses: google-github-actions/auth@v3
  with:
    project_id: ${{ env.PROJECT_ID }}
    workload_identity_provider: ${{ secrets.WIF_PROVIDER }}
    service_account: ${{ secrets.GCP_SERVICE_ACCOUNT }}

- name: Push image to Artifact Registry
  run: |
    docker push "${{ env.REGION }}-docker.pkg.dev/${{ env.PROJECT_ID }}/${{ env.REPO }}/${{ env.PACKAGE }}:${{ github.sha }}"

- name: Deploy to Cloud Run
  uses: google-github-actions/deploy-cloudrun@v3
  with:
    service: ${{ env.SERVICE }}
    image: "${{ env.REGION }}-docker.pkg.dev/${{ env.PROJECT_ID }}/${{ env.REPO }}/${{ env.PACKAGE }}:${{ github.sha }}"
    region: ${{ env.REGION }}
    env_vars: |
      KEDRO_ENV=prod
      MLFLOW_TRACKING_URI=${{ secrets.MLFLOW_TRACKING_URI }}
      MLFLOW_EXPERIMENT_NAME=${{ secrets.MLFLOW_EXPERIMENT_NAME }}
      LOG_LEVEL=INFO

```

Explanation

- `permissions.id-token: write` is required to mint an OIDC token for Workload Identity Federation.
- `google-github-actions/auth@v3` logs into GCP using the workload identity provider and service account.
- `docker build` builds the image. The tag uses `${{ github.sha }}` for immutability.
- `docker push` pushes the image to Artifact Registry.
- `deploy-cloudrun@v3` deploys the pushed image. You can pass environment variables as a multiline string.
- Define GitHub repository **Secrets**: `GCP_PROJECT_ID`, `GCP_REGION`, `GAR_REPO`, `CLOUD_RUN_SERVICE`, `WIF_PROVIDER` (the provider resource name), `GCP_SERVICE_ACCOUNT` (service account email), `MLFLOW_TRACKING_URI`, `MLFLOW_EXPERIMENT_NAME`.

11. Runtime configuration & secrets

Never store secrets in code or in the container image. Instead:

- Use Cloud Run **environment variables** for non-secret configuration (e.g., `KEDRO_ENV`, `LOG_LEVEL`).
- Store sensitive values (database credentials, MLflow tokens) in **Secret Manager** and mount them as environment variables using Cloud Run's secret reference. Example: in the `deploy-cloudrun` action, you can specify `secret_envs: MLFLOW_TOKEN=projects/<PROJECT_ID>/secrets/mlflow-token/versions/latest` to mount the token as `MLFLOW_TOKEN`.
- Provide the MLflow tracking URI and experiment name via environment variables. MLflow uses `MLFLOW_TRACKING_URI` to determine the tracking server ²⁰.

12. Smoke testing

After deployment, test the API locally and on Cloud Run.

Local

Run the API in Docker:

```
# build the image locally
docker build -t kedro-api:dev .
# run the container
PORT=8000 docker run -p 8000:8000 kedro-api:dev
# health check
curl http://localhost:8000/health
# run prediction
curl -X POST http://localhost:8000/run -H "Content-Type: application/json" -d
'{"x_new": [5, 7, 9]}'
```

Cloud Run

After deployment, Cloud Run prints a service URL. Test it with `curl`:

```
URL=$(gcloud run services describe <SERVICE> --platform managed --region
<REGION> --format='value(status.url)')
# health check
curl "$URL/health"
# run prediction
curl -X POST "$URL/run" -H "Content-Type: application/json" -d
'{"x_new": [5, 7, 9]}'
```

If authentication is required, add
 Authorization: Bearer \$(gcloud auth print-identity-token) or an IAM token.

13. Troubleshooting tips

Problem	Likely Cause / Fix
Service not ready / 503 errors	Ensure the app listens on <code>0.0.0.0</code> and uses the <code>PORT</code> env variable ² <code>3</code> .
Authentication errors in GitHub Actions	Check that the service account has <code>roles/run.admin</code> and <code>roles/artifactregistry.writer</code> and that the Workload Identity Provider ID matches your pool ¹⁴ . Ensure <code>permissions.id-token: write</code> is set in the workflow.
KedroSession cannot find configuration	Make sure <code>conf/</code> is copied into the Docker image and set <code>KEDRO_ENV</code> to the correct environment.
MLflow runs not appearing	Verify <code>MLFLOW_TRACKING_URI</code> and <code>MLFLOW_EXPERIMENT_NAME</code> environment variables. Ensure the service has network access and credentials for the tracking server.
Large payloads cause timeouts	Cloud Run has a request timeout (default 300 s). For long pipelines, increase <code>--timeout</code> in the deploy command or break work into async tasks.
Import errors	Confirm that <code>src/</code> is in the Python path. In the Dockerfile, set <code>PYTHONPATH=/app/src</code> or install the package with <code>pip install -e .</code>
Docker build slow	Use dependency caching (e.g., copy only <code>pyproject.toml</code> before installing dependencies) and a minimal base image.

14. One-page cheatsheet

Topic	Command / Snippet
Create venv & install	<code>uv venv .venv && source .venv/bin/activate && uv add kedro fastapi "uvicorn[standard]" mlflow</code>
Run pipeline	<code>with KedroSession.create(package_name="<pkg>", env="prod") as s: outputs = s.run(pipeline_name="example", extra_params={"x_new": [...]}))</code>
Run API locally	<code>uvicorn <pkg>.api.main:app --host 0.0.0.0 --port 8000</code>
Lint & format	<code>uv run ruff check src/</code> and <code>uv run black --check src/</code>
Docker build	<code>docker build -t \${REGION}-docker.pkg.dev/\${PROJECT_ID}/\${REPO}/\${pkg}:v1 .</code>

Topic	Command / Snippet
Docker run	<code>PORT=8000 docker run -p 8000:8000 image:tag</code>
Enable GCP APIs	<code>gcloud services enable artifactregistry.googleapis.com run.googleapis.com secretmanager.googleapis.com</code>
Create Artifact Registry	<code>gcloud artifacts repositories create <REPO> --repository- format=docker --location=<REGION></code> ¹¹
Push image	<code>docker push \${REGION}-docker.pkg.dev/\${PROJECT_ID}/\${REPO}/ <pkg>:tag</code>
Deploy Cloud Run	<code>gcloud run deploy <SERVICE> --image \${REGION}-docker.pkg.dev/\${ PROJECT_ID}/\${REPO}/<pkg>:tag --region <REGION> --platform managed --allow-unauthenticated</code> ¹³
Create WIF pool	<code>gcloud iam workload-identity-pools create github --project \$ {PROJECT_ID} --location=global --display-name "GitHub Actions Pool"</code> ¹⁵
Create WIF provider	<code>gcloud iam workload-identity-pools providers create-oidc github- provider --project \${PROJECT_ID} --location=global --workload- identity-pool=github --display-name "GitHub provider" -- attribute-mapping=... --attribute- condition="assertion.repository_owner=='<org>'" --issuer- uri="https://token.actions.githubusercontent.com"</code> ¹⁷
Grant SA roles	<code>gcloud projects add-iam-policy-binding \${PROJECT_ID} --member serviceAccount:SA@\${PROJECT_ID}.iam.gserviceaccount.com --role roles/run.admin and roles/artifactregistry.writer</code>
Bind WIF to SA	<code>gcloud iam service-accounts add-iam-policy-binding SA@\$ {PROJECT_ID}.iam.gserviceaccount.com --role roles/ iam.workloadIdentityUser --member "principalSet:// iam.googleapis.com/\${WORKLOAD_IDENTITY_POOL_ID}/ attribute.repository/<org>/<repo>"</code> ¹⁸
Retrieve Cloud Run URL	<code>gcloud run services describe <SERVICE> --platform managed -- region <REGION> --format='value(status.url)'</code>
Test API	<code>curl -X POST "\$URL/run" -H 'Content-Type: application/json' -d '{"x_new": [1, 2]}'</code>

This end-to-end workflow demonstrates how to take a Kedro project, wrap it with a FastAPI interface, package it in a Docker container, and deploy it to Google Cloud Run with automated CI/CD. Using **uv** for dependency management, **ruff** and **black** for code quality, and **MLflow** for experiment tracking ensures a reproducible and maintainable production ML service.

1 [kedro.framework.session.session.KedroSession](https://docs.kedro.org/en/0.18.14/kedro.framework.session.session.KedroSession.html) — kedro 0.18.14 documentation

<https://docs.kedro.org/en/0.18.14/kedro.framework.session.session.KedroSession.html>

2 Cloud Run revision fails because container still listens on 127.0.0.1 and there's no flag to override host · Issue #832 · google/adk-python

<https://github.com/google/adk-python/issues/832>

3 The Simplest Way to Make Cloud Run Port Work Like It Should

<https://hoop.dev/blog/the-simplest-way-to-make-cloud-run-port-work-like-it-should/>

4 Managing dependencies | uv

<https://docs.astral.sh/uv/concepts/projects/dependencies/>

5 Running commands | uv

<https://docs.astral.sh/uv/concepts/projects/run/>

6 10 Tutorial | Ruff

<https://docs.astral.sh/ruff/tutorial/>

7 The basics - Black 26.1.0 documentation

https://black.readthedocs.io/en/stable/usage_and_configuration/the_basics.html

8 Lifecycle management with KedroSession — kedro 0.18.14 documentation

https://docs.kedro.org/en/0.18.14/kedro_project_setup/session.html

9 Hooks examples -

<https://docs.kedro.org/en/stable/extend/hooks/examples/>

11 12 13 Cloud Run: A Guide to Deploying Containerized Apps in GCP | DataCamp

<https://www.datacamp.com/tutorial/cloud-run>

14 GitHub - google-github-actions/deploy-cloudrun: A GitHub Action for deploying services to Google Cloud Run.

<https://github.com/google-github-actions/deploy-cloudrun>

15 16 17 18 19 GitHub - google-github-actions/auth: A GitHub Action for authenticating to Google Cloud.

<https://github.com/google-github-actions/auth>

20 mlflow.environment_variables

https://mlflow.org/docs/latest/api_reference/python_api/mlflow.environment_variables.html